

The Founder's Guide to What We Built

What the machine is, how sound it is, what to do when something breaks, and how to work with any AI model on it — written for a non-technical owner.

1 • How sound is it — the honest verdict

Verdict: a well-built foundation that has not yet faced the real world.

Strong on: code quality, structure, automated testing (22 unit + 28 end-to-end checks against a real database), and security basics — passwords stored the way banks do it, secret keys kept out of files entirely.

The honest caveat: no card has ever appeared on a real phone yet. Until the Apple and Google credentials exist and we run the live test, “tested” means “tested everywhere except the last mile.” That last mile is blocked on account approvals, not code.

2 • The machine, in one picture

Stampy is three pieces talking to each other:

The card — lives in the customer's phone wallet

Shows stamps, a QR code, and the reward. Updates itself. Customers never download an app.

The staff page — any phone's browser, PIN-protected

Scan the customer's card (or type its 6-letter code), then +1 stamp, redeem, or send a message.

The brain — Railway (a rented server + database)

Remembers every card and stamp, and tells Apple or Google to update the customer's phone.

The journey of one stamp

1. Customer shows their wallet card; staff scans it (or types the code).
2. The brain writes +1 stamp into the database. This is the moment the stamp is safe, whatever happens next.
3. The brain tells Apple or Google “this card changed.”
4. The customer's phone updates the card and shows a notification, usually within seconds.

Why most failures are low-drama: once the stamp is in the database (step 2), a hiccup in steps 3–4 only delays the phone’s notification. The stamp itself is never lost.

3 • If something goes wrong

Almost every problem is one of these. None of them lose data.

Staff page won’t load at all

The server (Railway) is down or the bill lapsed. Check Railway’s dashboard. Cards already in wallets are untouched; stamping resumes when it’s back. At the counter, worst case: note the card code on paper and stamp later.

Stamp works, but no notification on the phone

An Apple/Google messaging hiccup, or the customer muted the pass. Do nothing — the stamp is saved and the card catches up on its own. If it never fires for anyone, that’s a certificate problem; tell your AI assistant.

“Almost ready” page instead of the card button

A credential is missing or expired. Open /setup — it lists in red exactly what’s missing.

Everything breaks about a year from now

Apple certificates expire annually. Diarize it: renew the Apple certificates about 11 months after creating them. Ten minutes of clicking, same walkthrough as setup.

Someone unwanted is stamping

The staff PIN leaked. Change it in the dashboard; it takes effect immediately.

You forget the dashboard password

Honest gap: there’s no “forgot password” yet. An AI session can reset it in the database. It’s on the fix list.

Your three windows into the machine

- /setup — the traffic-light page. All green = healthy. Always look here first.
- /health — one line that says ok:true if the server is alive.
- Railway’s dashboard — shows whether the app is running and what it’s spending. Its “Logs” tab is what you screenshot for an AI assistant when something’s weird.

4 • What you actually need to know

Just these four things. You never need to open the code.

The stamp is safe the moment it’s saved

Staff scans, the database records it, then the phone gets notified. So a glitch delays the notification but never loses a stamp.

/setup is your traffic-light page

Green means healthy; red tells you exactly what's missing, in plain English. That plus Railway's dashboard covers almost every "what's going on?"

One calendar item: Apple certificates expire yearly

Diarize a renewal about 11 months out, or cards stop updating. Ten minutes when the time comes.

The lie-detector command

You can't read code and you don't need to. Any time any AI model changes anything, make it prove the change is safe:

run typecheck, all tests, and the e2e suite — show me the results

All green = the change didn't break anything the tests cover. Any red = "fix it before moving on." This one habit is 90% of supervising AI work without reading code.

5 • Working with AI models on this code

Models will come and go. Two things protect you across all of them.

The repo is the memory

Everything a fresh model needs is in the project itself: the README explains the system, a CLAUDE.md file lists the untouchable rules, and the tests define "working." You never re-explain the product from scratch — just point any model at the folder.

House rules now written into the repo

- Secrets live in Railway's website only — never in files, never in the code.
- The app must always boot even with zero credentials (that's why /setup can exist).
- Exactly one notification per event — Google hard-caps at 3 per day; on Apple, more would turn one stamp into spam.
- Save a checkpoint after every working change, so anything can be undone.

6 • What's deliberately not built yet

Known gaps, on the list, in rough order: a database backup schedule on Railway (one click at deploy time), a "forgot password" flow, uptime alerts (an email if the app goes down), billing for cafés, and the automated win-back scheduler. None block the pilot; all should exist before real cafés pay you.

Bottom line

The code is in far better shape than most weekend projects — typed, tested, and structured for the multi-café future. Its one real debt is verification on physical phones, and that's waiting on Apple and Google paperwork, not on engineering.